1. **Challenge Title:** "Guardian Satellite - Decentralized Health Monitoring System"

The Problem

Space agencies and satellite operators face critical challenges in monitoring satellite health in real-time. When satellites malfunction, delayed detection can result in:

- Loss of communication services affecting millions
- Failure of weather forecasting systems
- Disruption of GPS navigation
- Costly mission failures

Your Mission: Build a satellite health monitoring system that tracks multiple satellite components, predicts failures before they occur, and uses blockchain technology to create an immutable record of satellite health data that can be trusted by multiple stakeholders (space agencies, insurance companies, satellite operators).

System Components to Monitor

Your system should track these 5 key satellite subsystems:

1. **Power System (Solar Panels & Batteries)**
    - Battery voltage levels
    - Solar panel efficiency
    - Power consumption rate
2. **Thermal Control System**
    - Temperature of critical components
    - Radiator performance
    - Heater status
3. **Communication System**
    - Signal strength
    - Antenna orientation
    - Data transmission rate
4. **Attitude Control (Orientation)**
    - Gyroscope readings
    - Reaction wheel speed
    - Satellite pointing accuracy

5. **Onboard Computer**
    - CPU usage
    - Memory availability
    - Software errors/crashes

Core Features to Build

1. Real-Time Monitoring Dashboard (Frontend)

- Display live status of all 5 subsystems
- Color-coded health indicators (Green/Yellow/Red)
- Simple graphs showing trends

2. Smart Alarm System (Algorithm)

Create an algorithm that triggers alarms when:

- **Critical**: Immediate danger (e.g., battery < 20%, temperature > 85°C)
- **Warning**: Potential issue (e.g., battery draining faster than normal)
- **Anomaly**: Unusual patterns detected

3. Web3 Integration (Blockchain)

- Store critical health events on blockchain (immutable record)
- Create NFT-based maintenance certificates
- Smart contract that automatically triggers alerts to stakeholders

4. Predictive Analytics (Bonus)

- Predict when components might fail based on trends
- Suggest maintenance schedules

Round 1 constraints:

1. Mission Objectives & Success Criteria

Every design must start with the **"Why."** In a hackathon, your mission objective isn't just a goal; it's a boundary.

- **Operational Lifetime:** How long must the satellite survive? (e.g., 6 months for a tech demo vs. 5 years for a weather satellite).
- **Target Orbit:** This dictates almost everything else.
    - **LEO (Low Earth Orbit):** Lower latency, higher resolution, but high atmospheric drag.

    - **GEO (Geostationary Orbit):** Constant coverage of one area, but extremely high launch costs and radiation.

- **Data Latency:** Does the data need to be "real-time" (for disaster relief) or can it be stored and forwarded once a day?

2. Payload Constraints

The payload is the "customer" of the satellite; everything else (power, thermal, comms) exists just to keep it happy.

- **SWaP-C:** This is the golden rule of satellite design: **S**ize, **W**eight, **a**nd **P**ower - **C**ost.
- **Mass & Volume:** Most hackathons use the "U" standard (1U = 10x10x10 cm). You'll likely be restricted to a 3U or 6U form factor.
- **Duty Cycle:** Can the payload run 24/7, or does it need to shut down to recharge batteries while the satellite is in the Earth's shadow?

3. Budget & Resource Estimation

In a real-world scenario (and a good hackathon), "unlimited money" isn't a thing.

- **Total Cost of Ownership (TCO):** You must account for:
    1. **Development:** Components and labor.
    2. **Launch:** Often calculated per kilogram (e.g., $50,000/kg for LEO).
    3. **Operations:** Ground station fees to talk to your satellite.
- **COTS vs. Custom:** Using **Commercial Off-The-Shelf (COTS)** parts is cheaper and faster but might not be "space-hardened" against radiation compared to custom, radiation-hardened components.

4. Technical Constraints & Requirements

1. Data Efficiency Constraint (The "Warning-Only" Rule)

Space-to-ground bandwidth is expensive. You cannot log every second of "healthy" data to the blockchain or permanent record.

- **Requirement:** Your system must only trigger a "Write to Blockchain/Log" action when a subsystem moves into a **Warning (Yellow)** or **Critical (Red)** state.
- **Success Criteria:** The permanent event log should stay empty while the satellite is "Green," demonstrating smart data management.

2. Power Save Constraint (The "Eclipse" Protocol)

When a satellite passes into the Earth's shadow, it relies on limited battery power.

- **Requirement:** If the simulated **Battery Level drops below 25%**, the system must automatically enter **"Low Power Mode."**
- **Success Criteria:** In this mode, the UI must "grey out" or disable non-essential telemetry (e.g., CPU Usage and Communication Rates) to prioritize core survival data (Power and Thermal).

3. The "Human-in-the-Loop" Verification

For insurance and safety purposes, critical failures cannot be "self-cleared" by the satellite's computer alone.

- **Requirement:** When a **Critical (Red)** alarm is triggered, the status cannot return to "Green" automatically, even if the sensors improve.
- **Success Criteria:** A **"Manual Acknowledge" button** must be pressed by a human operator to verify the event and clear the alert from the dashboard.